

MediBook Phase 4

Booking System with Concurrency Control

COMPLETE LEARNING GUIDE

Pessimistic Locking · Optimistic Locking
Race Conditions · Audit Trails · State Machines
Booking References · Idempotency

What You Built in Phase 4

BookingStatus + BookingHistoryAction enums
Booking entity with @Version (optimistic lock)
BookingHistory entity (immutable audit trail)
Repository with @Lock PESSIMISTIC_WRITE
DR{doctor}-{date}-{sequence} reference generator
Concurrency-safe BookingServiceImpl
2-hour cancellation rule (role-based)
N+1 query fix with batch loading
18 endpoints across 3 controllers
Defense-in-depth validation
Resource hiding (404 vs 403)
End-to-end Postman smoke tests passing

Stack: Java 21 · Spring Boot 3.5 · MySQL 8 · JPA · Spring Security

Repository: github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook

Table of Contents

PART I CONCURRENCY DEEP DIVE

1. The Race Condition Problem
2. Pessimistic Locking Explained
3. Optimistic Locking Explained
4. When to Use Which (and Both)

PART II DESIGN DECISIONS

5. Business Requirements
6. State Machine: BookingStatus
7. Booking Reference Format (DR5-...)
8. Cancellation Policy (2-Hour Rule)
9. Rich Audit Trail (BookingHistory)

PART III DATA LAYER

10. BookingStatus & BookingHistoryAction Enums
11. Booking Entity Deep Dive
12. BookingHistory Entity (Append-Only)
13. BookingRepository with @Lock
14. BookingHistoryRepository

PART IV DTO & MAPPING

15. 7 DTOs Overview
16. Two Response Variants (Full & Summary)
17. Composite Detail+History Response

PART V SERVICE LAYER

18. BookingReferenceGenerator
19. BookingServiceImpl - The Critical Code
20. Role-Based Cancellation Rules
21. BookingHistoryService (N+1 Fix)

PART VI CONTROLLERS & SECURITY

22. Three Controllers Pattern
23. Ownership Verification (Resource Hiding)
24. SecurityConfig Strategy

PART VII TESTING

25. End-to-End Setup Journey
26. Smoke Test 1: Create Booking
27. Smoke Test 2: Reference Format
28. Smoke Test 3: Cancellation Flow
29. Smoke Test 4: Doctor Completes
30. Smoke Test 5: View History
31. Common Errors & Solutions

PART VIII REFERENCE

32. All 18 Endpoints Cheat Sheet
33. Interview Q&A; for Phase 4
34. Phase 5 Preview: Async Notifications

PART I · CONCURRENCY DEEP DIVE

1. The Race Condition Problem

The booking system is the most concurrency-critical part of MediBook. When multiple patients try to book the same slot simultaneously, we **MUST** ensure data integrity. Without proper safeguards, race conditions cause oversubscription and silent data corruption.

The Classic Bug (Step by Step)

Race condition timeline

Setup: slot has max_bookings=2, current_bookings=1 (one spot left).

TIME	PATIENT A	PATIENT B
0 ms	Reads slot (current=1)	
1 ms		Reads slot (current=1)
2 ms	Checks: $1 < 2$ OK	
3 ms		Checks: $1 < 2$ OK
4 ms	UPDATE current=2 Save booking	
5 ms		UPDATE current=2 Save booking

RESULT:

- 2 bookings created (correct so far)
- current_bookings = 2 (not 3 - second UPDATE overwrote)
- Slot looks healthy but is OVERSUBSCRIBED
- Data corrupted silently

WARNING » Real companies have lost millions to bugs like this. Airlines overbook flights, ticketing sites sell phantom seats, banks duplicate transactions. The solution is explicit concurrency control.

2. Pessimistic Locking Explained

Philosophy: 'I expect conflicts. Let me lock the row so nobody else can touch it.'

How It Works

Pessimistic lock in action

TIME	PATIENT A	PATIENT B
0 ms	SELECT ... FOR UPDATE Lock acquired on slot row	
1 ms		SELECT ... FOR UPDATE WAITS for lock (blocked)
2 ms	Checks: 1 < 2 OK UPDATE current=2 Save booking	
3 ms	COMMIT Lock released	
4 ms		(unblocked!) Reads slot (current=2) Checks: 2 < 2 FAIL Throws "Slot is full"

RESULT:

- Only 1 booking succeeded (correct!)
- Second patient got clear error
- Database integrity preserved

Implementation in Our Repository

BookingRepository @Lock method

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@QueryHints({
    @QueryHint(name = "jakarta.persistence.lock.timeout", value = "5000")
})
@Query("SELECT s FROM DoctorSlot s WHERE s.id = :id")
Optional<DoctorSlot> findByIdWithLock(@Param("id") Long id);
```

Generated SQL

What JPA generates

```
SELECT * FROM doctor_slots WHERE id = ?
FOR UPDATE;                -- The magic clause
```

Why this matters » FOR UPDATE tells the database: 'Lock this row exclusively until my transaction ends.' Other transactions trying to SELECT or UPDATE this row will BLOCK until our COMMIT. This is the strongest concurrency guarantee available.

The Critical Timeout

Always set **jakarta.persistence.lock.timeout**. Without it, a transaction can wait indefinitely for a lock, exhausting connection pools and bringing down your service. 5 seconds is a good default - long enough for normal contention, short enough to fail fast on deadlocks.

3. Optimistic Locking Explained

Philosophy: 'I expect conflicts to be rare. Let me detect them at save time.'

How It Works

Optimistic lock in action

Every row has a hidden 'version' field. JPA auto-increments on every UPDATE.

TIME	PATIENT A	PATIENT B
0 ms	SELECT * (version=5)	
1 ms		SELECT * (version=5)
2 ms	UPDATE current=2, version=6 WHERE id=? AND version=5 Result: 1 row affected (success) COMMIT	
3 ms		UPDATE current=2, version=6 WHERE id=? AND version=5 Result: 0 rows affected (FAIL) JPA throws OptimisticLockException

RESULT:

- A succeeded
- B got an exception
- App can retry B with fresh data

Implementation on Entity

Booking entity field

```
@Version
@Column(name = "version", nullable = false)
private Long version = 0L;
```

That's it. JPA does all the magic. Every UPDATE generates SQL with WHERE version=? and JPA increments the version. If the WHERE doesn't match, throws OptimisticLockException.

Generated SQL

What JPA generates

```
UPDATE bookings
SET status = 'CANCELLED', version = 6 -- auto-incremented
WHERE id = ? AND version = 5;        -- auto-checked
```

4. When to Use Which (and Both)

Aspect	Pessimistic	Optimistic
Philosophy	Prevent conflicts	Detect conflicts
DB locks	Yes (row lock)	No (lock-free)
Other threads	WAIT	Proceed, fail at save
Throughput	Lower under contention	High
Use case	Hot rows, critical paths	General updates
Failure mode	Wait then proceed	Exception then retry
Our use case	Booking critical section	Booking row updates

Our Strategy: BOTH

Defense in depth

DoctorSlot entity has @Version (Phase 3)

Booking entity has @Version (Phase 4)

BookingRepository.findByIdWithLock() uses @Lock PESSIMISTIC_WRITE

WHY BOTH?

Pessimistic lock on slot during critical sections:

- Prevents the booking storm scenario
- Serializes access to hot rows

Optimistic @Version on general updates:

- Safe default for entity modifications
- Catches concurrent updates outside the critical path

INTERVIEW POINT » Interview Q: Explain pessimistic vs optimistic locking and when to use which.

A: Both prevent concurrent modifications but differ in strategy. Pessimistic uses @Lock(PESSIMISTIC_WRITE) which generates FOR UPDATE SQL clauses, acquiring exclusive row locks at SELECT time. Other transactions wait until commit. It's safer for high-contention scenarios but reduces throughput. Optimistic uses @Version - JPA increments a version field on each update and checks it in WHERE clauses. Concurrent updates that miss the version throw OptimisticLockException for application-level retry. Pessimistic for hot rows where preventing conflicts is cheaper than detecting them; optimistic for general entity updates where conflicts are rare. In my booking system I use both: pessimistic on slot rows during booking critical section, optimistic @Version on Booking and DoctorSlot entities for general updates.

PART II · DESIGN DECISIONS

5. Business Requirements

Functional Requirements

- Patients can book any AVAILABLE slot from any APPROVED doctor
- Patients can have multiple bookings (different doctors/times)
- Same patient + same slot is forbidden (UNIQUE constraint)
- Cancellation allowed (with 2-hour rule for patients only)
- Doctors can mark bookings COMPLETED or NO_SHOW after slot time
- Admins can override anything (cancel, view, manage)
- Rich audit trail for compliance

Non-Functional Requirements

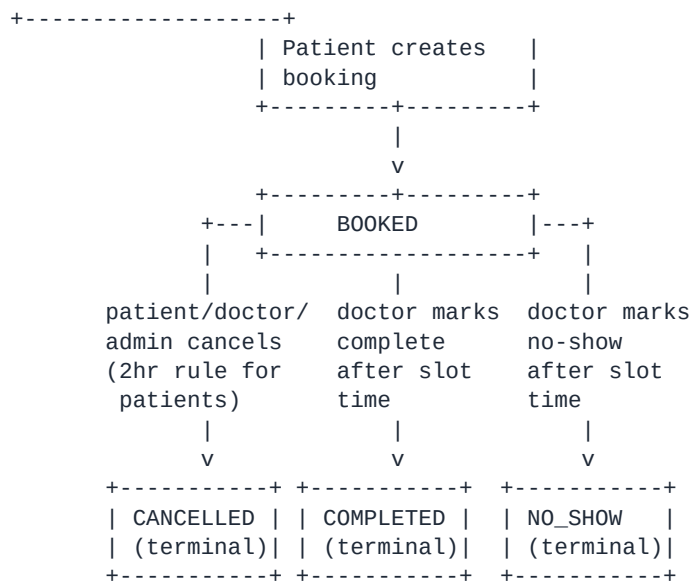
- **Concurrency:** No race conditions, no oversubscription
- **Atomicity:** Each booking action either fully succeeds or fully rolls back
- **Audit:** Every state change captured immutably
- **Security:** Patients see only their bookings, doctors only their slots
- **Performance:** Sub-100ms booking creation

Locked Design Choices

Choice	Decision	Why
Cancellation policy	2-hour rule for patients	Industry standard, balances flexibility & doctor protection
Multi-bookings	Allowed (different slots)	Patients may need multiple specialists same day
Reference format	DR5-20261209-001	Self-descriptive, sortable, debuggable
Audit depth	Rich (every change)	Compliance, analytics, forensics

6. State Machine: BookingStatus

Booking lifecycle



ENFORCED IN SERVICE:

- createBooking only creates BOOKED status
- cancel/complete/noShow only allowed from BOOKED
- Three terminal states - no further transitions

Why this matters » Explicit state machines prevent invalid transitions. The service layer enforces which transitions are allowed from each state. Trying to 'complete' a CANCELLED booking throws `BadRequestException` immediately. This eliminates entire categories of bugs that boolean flags (`isCancelled`, `isCompleted`) can't catch.

7. Booking Reference Format

The Format: DR5-20261209-001

Reference anatomy

```
DR5-20261209-001
  ^      ^      ^
  |      |      +-- 3-digit sequence (001 to 999)
  |      +----- Date in yyyyMMdd format
  +----- DR + doctor ID
```

Examples:

```
DR1-20261225-001 -- First booking for Doctor 1 on Christmas 2026
DR12-20261225-042 -- 42nd booking for Doctor 12 same date
DR5-20261210-001 -- First booking for Doctor 5 next day
```

PROPERTIES:

```
Sortable: ORDER BY booking_reference works correctly
Scannable: Humans can read it
Unique: Sequence within (doctor, date)
Stable: Never changes once assigned
Compact: ~20 characters fits in URLs/SMS
```

Why Not UUID?

UUIDs are opaque - 'a7f3e9c1-7425-40de-944b-e07fc1f90ae7' tells you nothing. Our format conveys business meaning: doctor, date, sequence. Support staff and patients can read it. The trade-off is concurrency handling for the sequence (which we solve with retry logic in `BookingReferenceGenerator`).

8. Cancellation Policy (2-Hour Rule)

The Rule

Cancellation rule

Slot time: 10:00 AM

Cancellation cutoff: 8:00 AM (2 hours before)

Patient cancels at 7:59 AM -> Allowed

Patient cancels at 8:01 AM -> BLOCKED with error:

"Cannot cancel within 2 hours of the appointment.
Please contact the clinic directly."

ROLE-BASED EXCEPTIONS:

Doctors: No time restriction (medical emergencies)

Admins: No time restriction (system overrides)

Only PATIENTS are subject to the 2-hour rule.

Implementation

Role-based rule enforcement

```
private static final int CANCELLATION_HOURS_LIMIT = 2;

if ("PATIENT".equals(cancelledByRole)) {
    validatePatientCancellationWindow(booking);
}

private void validatePatientCancellationWindow(Booking booking) {
    LocalDateTime slotDateTime = booking.getSlotDateTime();
    LocalDateTime cutoff = slotDateTime.minusHours(CANCELLATION_HOURS_LIMIT);

    if (LocalDateTime.now().isAfter(cutoff)) {
        throw new BadRequestException(
            "Cannot cancel within " + CANCELLATION_HOURS_LIMIT +
            " hours of the appointment. Please contact the clinic directly.");
    }
}
```

9. Rich Audit Trail (BookingHistory)

Append-Only Pattern

Each Booking state change creates an immutable BookingHistory row. Rows are NEVER updated or deleted - they're permanent records of what happened.

Complete Booking Lifecycle

Lifecycle data flow

Day 1, 10:00 AM - Patient creates booking

```
Booking row INSERTED:  status = BOOKED
History row INSERTED:  action = CREATED
                        old_status = null
                        new_status = BOOKED
                        performed_by = patient_id
```

Day 2, 8:00 AM - Patient cancels

```
Booking row UPDATED:  status = CANCELLED
                      cancellation_reason = "..."
History row INSERTED:  action = CANCELLED
                      old_status = BOOKED
                      new_status = CANCELLED
                      performed_by = patient_id
                      notes = "Schedule conflict"
```

FINAL STATE:

```
Bookings:             1 row (CANCELLED)
BookingHistory:       2 rows (CREATED + CANCELLED)
```

TIP » Both old_status and new_status are stored on each history row. Slight denormalization (could derive old from previous row's new) but makes queries trivial: 'all CANCELLED transitions' = simple WHERE clause instead of LAG window function.

PART III · DATA LAYER

10. Two Critical Enums

BookingStatus (states - nouns)

Status enum

```
public enum BookingStatus {  
    BOOKED,           // Active reservation  
    COMPLETED,       // Patient attended, doctor finished  
    CANCELLED,        // Cancelled before slot time  
    NO_SHOW           // Patient didn't attend  
}
```

BookingHistoryAction (events - verbs)

Action enum

```
public enum BookingHistoryAction {  
    CREATED,           // Initial booking  
    CANCELLED,         // Booking cancelled  
    COMPLETED,        // Marked complete  
    NO_SHOW_MARKED     // Marked no-show  
}
```

Why this matters » Two enums for two different concepts. Status answers 'what state is the entity in NOW?' - the noun. Action answers 'what event just happened?' - the verb. They typically align (CREATED action sets status=BOOKED), but can diverge in future (e.g., NOTES_UPDATED action without changing status). Single Responsibility Principle applied to enum design.

11. Booking Entity Deep Dive

Booking schema

BOOKINGS		
id	BIGINT PK AUTO	
patient_user_id	BIGINT FK NOT NULL	-> users.id
slot_id	BIGINT FK NOT NULL	-> doctor_slots.id
booking_reference	VARCHAR(30) UNIQUE NOT NULL	DR5-20261209-001
status	VARCHAR(20) NOT NULL	enum string
notes	TEXT NULLABLE	
cancellation_reason	TEXT NULLABLE	
cancelled_by_user_id	BIGINT NULLABLE	
cancelled_at	TIMESTAMP NULLABLE	
completed_at	TIMESTAMP NULLABLE	
version	BIGINT DEFAULT 0	@Version
created_at, updated_at	TIMESTAMP	
UNIQUE (patient_user_id, slot_id)		
UNIQUE (booking_reference)		
INDEX (patient_user_id, status)		
INDEX (slot_id, status)		
INDEX (status, created_at)		

Critical Design Decisions

- **patient_user_id instead of patient_id:** We don't have a Patient entity yet - all users are stored in 'users' with role=PATIENT. Direct user reference is simpler.
- **UNIQUE (patient_user_id, slot_id):** Database-level idempotency. Even if a bug bypasses validation, DB rejects duplicates.
- **booking_reference as VARCHAR:** Stored, not computed. Indexed for fast lookup, sortable, business-meaningful.
- **Three composite indexes:** Designed for the three main query patterns (patient bookings, slot bookings, analytics).
- **@Version field:** Optimistic locking for general updates.

12. BookingHistory Entity (Append-Only)

BookingHistory schema

BOOKING_HISTORY		
id	BIGINT PK AUTO	
booking_id	BIGINT FK NOT NULL	
old_status	VARCHAR(20) NULLABLE	null for CREATED
new_status	VARCHAR(20) NOT NULL	
action	VARCHAR(20) NOT NULL	
performed_by_user_id	BIGINT NOT NULL	
notes	TEXT NULLABLE	
performed_at	TIMESTAMP NOT NULL	
INDEX (booking_id, performed_at)		timeline
INDEX (performed_by_user_id)		user audit
INDEX (action, performed_at)		analytics

Static Factory Methods (Clean Service Code)

Factory methods

```
public static BookingHistory created(Booking booking, Long performedByUserId) {
    return new BookingHistory(booking, null, BookingStatus.BOOKED,
        BookingHistoryAction.CREATED, performedByUserId, null);
}

public static BookingHistory cancelled(Booking booking, BookingStatus oldStatus,
    Long performedByUserId, String reason) {
    return new BookingHistory(booking, oldStatus, BookingStatus.CANCELLED,
        BookingHistoryAction.CANCELLED, performedByUserId, reason);
}

// Service code becomes:
BookingHistory history = BookingHistory.cancelled(booking, BookingStatus.BOOKED, userId, reason);
bookingHistoryRepository.save(history);
```

TIP » Factory methods make service code self-documenting. Reader sees 'BookingHistory.cancelled(...)' and knows exactly what's happening without scanning constructor arguments.

13. BookingRepository with @Lock

The Critical Method

Pessimistic lock with timeout

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@QueryHints({
    @QueryHint(name = "jakarta.persistence.lock.timeout", value = "5000")
})
@Query("SELECT b FROM Booking b WHERE b.id = :id")
Optional<Booking> findByIdWithLock(@Param("id") Long id);
```

Other Notable Methods

- **existsByPatientUserIdAndSlotIdAndStatusIn**: Idempotency check for re-booking
- **countByDoctorIdAndSlotDate**: Used by reference generator
- **existsByBookingReference**: Collision detection in generator
- **findByPatientUserAndStatusIn**: Patient's bookings by status
- **countActiveBookingsForSlot**: Real-time capacity check

14. BookingHistoryRepository

No Locks Needed (Append-Only Magic)

History rows are immutable. There are no UPDATE or DELETE operations, so write-write conflicts cannot occur. Concurrent INSERTs target different rows. This eliminates an entire category of concurrency bugs without explicit synchronization.

Read-Heavy Query Methods

Query methods

```
// Booking timeline (chronological)
List<BookingHistory> findByBookingIdOrderByPerformedAtAsc(Long bookingId);

// User audit log
Page<BookingHistory> findByPerformedByUserIdOrderByPerformedAtDesc(
    Long userId, Pageable pageable);

// Analytics with date range
long countByActionAndPerformedAtBetween(
    BookingHistoryAction action,
    LocalDateTime start,
    LocalDateTime end);

// Doctor-specific analytics
long countByActionAndDoctorAndDateRange(
    BookingHistoryAction action,
    Long doctorId,
    LocalDateTime start,
    LocalDateTime end);
```

PART IV · DTO & MAPPING

15. 7 DTOs Overview

DTO	Type	Purpose
CreateBookingRequest	Request	Patient books a slot
CancelBookingRequest	Request	Cancel with reason
CompleteBookingRequest	Request	Doctor marks complete (optional notes)
BookingResponse	Response	Full booking details for detail views
BookingSummaryResponse	Response	Compact for list views
BookingHistoryResponse	Response	Single audit entry
BookingDetailWithHistoryResponse	Response	Composite: booking + timeline

16. Two Response Variants (Full & Summary)

Two-DTO strategy

BookingResponse (~20 fields):

id
bookingReference
status
patientUserId, patientName,
patientEmail
slotId, slotDate, slotStart/End
doctorId, doctorName,
doctorSpecialization
notes
cancellationReason
cancelledByUserId, cancelledAt
completedAt
createdAt, updatedAt

USE CASES:

Detail view (1 booking)
Booking confirmation
Cancellation response

BookingSummaryResponse (~8 fields):

id
bookingReference
status
slotDate, slotStartTime, slotEndTime
doctorName
patientName

USE CASES:

List view (50+ bookings)
Patient's "My Bookings"
Doctor's daily roster

Why two? List view with 50 bookings:

Full DTO: ~100KB JSON, slow
Summary DTO: ~25KB JSON, 4x faster

17. Composite Detail+History Response

Composite response DTO

```
public class BookingDetailWithHistoryResponse {  
    private BookingResponse booking;  
    private List<BookingHistoryResponse> history;  
}
```

Sample Response

Combined data

```
{  
  "booking": {  
    "id": 42,  
    "bookingReference": "DR5-20261209-001",  
    "status": "CANCELLED",  
    ...  
  },  
  "history": [  
    { "action": "CREATED", "performedAt": "...", "newStatus": "BOOKED" },  
    { "action": "CANCELLED", "performedAt": "...", "newStatus": "CANCELLED",  
      "notes": "Schedule conflict" }  
  ]  
}
```

Why this matters » Frontend renders the booking card AND the timeline from one API call. No client-side N+1, no need for sequential calls. The detail page is fast to load.

PART V · SERVICE LAYER

18. BookingReferenceGenerator

The Algorithm

Reference algorithm

INPUTS: doctorId=5, slotDate=2026-12-09

STEP 1: Count existing bookings for this (doctor, date)
count = bookingRepository.countByDoctorIdAndSlotDate(5, 2026-12-09)
Example: count = 0

STEP 2: Compute next sequence
sequence = count + 1 = 1

STEP 3: Format
reference = "DR" + 5 + "-" + "20261209" + "-" + "001"
= "DR5-20261209-001"

STEP 4: Check for collision
if (existsByBookingReference("DR5-20261209-001")) {
 // Retry with sequence + 1
}

STEP 5: Return reference

Retry Logic for Concurrency

Concurrency-safe generation

```
for (int attempt = 0; attempt < MAX_RETRIES; attempt++) {  
    long existingCount = bookingRepository.countByDoctorIdAndSlotDate(doctorId, slotDate);  
    long sequence = existingCount + 1 + attempt;    // Skip retried sequences  
  
    if (sequence > MAX_SEQUENCE) {  
        throw new IllegalStateException("Exceeded max sequence");  
    }  
  
    String candidate = buildReference(doctorId, dateString, sequence);  
  
    if (!bookingRepository.existsByBookingReference(candidate)) {  
        return candidate;  
    }  
    log.warn("Collision on attempt {} for {} - retrying", attempt, candidate);  
}
```

WARNING » Under simultaneous bookings, two threads might compute the same sequence. The DB UNIQUE constraint catches it; we retry with the next sequence number. MAX_RETRIES=5 - in practice even with thousands of concurrent bookings, retry is needed only rarely.

19. BookingServiceImpl - The Critical Code

createBooking Flow (THE CRITICAL ONE)

Booking flow with pessimistic locking

```
@Transactional
public BookingResponse createBooking(Long patientUserId, CreateBookingRequest request) {

    STEP 1: Look up patient (validate exists)

    STEP 2: ACQUIRE PESSIMISTIC LOCK on slot
        DoctorSlot slot = slotRepository.findByIdWithLock(slotId);
        --- Other transactions trying same slot WAIT HERE ---

    STEP 3: Validate slot state (FRESH data inside lock):
        - status == AVAILABLE
        - not expired (slot time still in future)
        - currentBookings < maxBookings

    STEP 4: Check duplicate booking
        existsByPatientUserIdAndSlotIdAndStatusIn(
            patientId, slotId, [BOOKED, COMPLETED])
        -> reject if true (allows re-book after CANCELLED)

    STEP 5: Generate booking reference (DR5-20261209-001)

    STEP 6: Create Booking entity (status=BOOKED)
        bookingRepository.save(booking);

    STEP 7: Increment slot capacity
        slot.setCurrentBookings(current + 1);
        if (current == max) slot.status = FULLY_BOOKED;
        slotRepository.save(slot);

    STEP 8: Audit trail
        BookingHistory.created(booking, patientId);
        bookingHistoryRepository.save(history);

} -- TRANSACTION COMMITS, lock released

ALL OR NOTHING: any failure rolls back EVERYTHING.
```

INTERVIEW POINT » Interview Q: Walk me through a concurrency-safe booking creation.

A: It's all inside one `@Transactional` method. First look up the patient. Then acquire a pessimistic write lock on the slot using `@Lock(PESSIMISTIC_WRITE)` - this serializes access at the database level. With the lock held, reload fresh slot data and validate: status `AVAILABLE`, not expired, has capacity. Check the patient doesn't already have an active booking for this slot. Generate the booking reference, create the `Booking` entity, increment `slot.currentBookings`, and create a `BookingHistory` entry. The transaction commits as a single atomic unit - either all succeed or all rollback. The lock releases on commit, allowing waiting transactions to proceed with the updated state. No race conditions, no oversubscription possible.

20. Role-Based Cancellation Rules

Role-aware cancellation

```
@Transactional
public BookingResponse cancelBooking(Long bookingId, CancelBookingRequest request,
                                     Long cancelledByUserId, String cancelledByRole) {
    Booking booking = bookingRepository.findByIdWithLock(bookingId)
        .orElseThrow(() -> new ResourceNotFoundException("Booking", "id", bookingId));

    if (!booking.isCancellable()) {
        throw new BadRequestException("Cannot cancel booking in state: " + booking.getStatus());
    }

    // Enforce 2-hour rule for PATIENT cancellations only
    if ("PATIENT".equals(cancelledByRole)) {
        validatePatientCancellationWindow(booking);
    }
    // Doctor and admin bypass the rule (medical emergencies, overrides)

    BookingStatus oldStatus = booking.getStatus();

    booking.setStatus(BookingStatus.CANCELLED);
    booking.setCancellationReason(request.getReason());
    booking.setCancelledByUserId(cancelledByUserId);
    booking.setCancelledAt(LocalDateTime.now());
    bookingRepository.save(booking);

    // Decrement slot capacity, restore AVAILABLE if was FULLY_BOOKED
    DoctorSlot slot = booking.getSlot();
    slot.setCurrentBookings(Math.max(0, slot.getCurrentBookings() - 1));
    if (slot.getStatus() == DoctorSlotStatus.FULLY_BOOKED) {
        slot.setStatus(DoctorSlotStatus.AVAILABLE);
    }
    slotRepository.save(slot);

    // Audit trail
    BookingHistory history = BookingHistory.cancelled(
        booking, oldStatus, cancelledByUserId, request.getReason());
    bookingHistoryRepository.save(history);

    return bookingMapper.toResponse(booking);
}
```

Why this matters » Single service method handles all three caller roles with different rules. Controllers pass the role explicitly. Avoids code duplication; centralizes business logic. Doctors can cancel for medical emergencies anytime; patients are restricted by the 2-hour rule.

21. BookingHistoryService (N+1 Fix)

The Problem (Without Batch Loading)

Without batch

List 100 history entries:

```
History Row 1: performed_by_user_id = 100
  -> Query: SELECT user WHERE id = 100 ("alice@example.com")
History Row 2: performed_by_user_id = 100
  -> Query: SELECT user WHERE id = 100 (AGAIN!)
History Row 3: performed_by_user_id = 200
  -> Query: SELECT user WHERE id = 200
...
```

TOTAL: 1 + 100 = 101 queries (N+1 problem)

TIME: ~500ms

The Fix (Batch Loading)

Batch loading pattern

```
private List<BookingHistoryResponse> enrichWithUserEmails(List<BookingHistory> historyList) {
    // STEP 1: Collect unique user IDs
    Set<Long> userIds = historyList.stream()
        .map(BookingHistory::getPerformedByUserId)
        .collect(Collectors.toSet());

    // STEP 2: ONE query for all users
    List<User> users = userRepository.findAllById(userIds);

    // STEP 3: Build lookup map
    Map<Long, String> userIdToEmail = users.stream()
        .collect(Collectors.toMap(User::getId, User::getEmail));

    // STEP 4: Map history entries using prefetched email
    return historyList.stream()
        .map(history -> toResponseWithEmail(history, userIdToEmail))
        .collect(Collectors.toList());
}
```

Performance gain

After batch loading:

TOTAL: 1 + 1 = 2 queries

TIME: ~10ms (50x faster!)

TIP » Use Set to deduplicate IDs before findAllById. 100 history rows might reference only 5 unique users; smaller IN clause = faster query.

PART VI · CONTROLLERS & SECURITY

22. Three Controllers Pattern

Controller	Role	Path Prefix	Endpoints
PatientBookingController	PATIENT	/me/bookings	7
DoctorBookingController	DOCTOR	/doctors/me/bookings	6
AdminBookingController	ADMIN	/admin/bookings	5

Same Pattern as Phase 2/3

Splitting by audience makes URL paths intuitive. **/me/bookings/...** is obviously self-service. **/doctors/me/bookings/...** is the doctor's view. **/admin/bookings/...** is admin operations. Recruiters or team members understand the API surface at a glance.

Class-Level @PreAuthorize

Class-level auth

```
@RestController
@RequestMapping("/me/bookings")
@PreAuthorize("hasRole('PATIENT')")
public class PatientBookingController { ... }
```

TIP » Class-level @PreAuthorize covers all methods. No per-method overrides needed. Single point of control.

23. Ownership Verification (Resource Hiding)

Ownership check in PatientBookingController

```
private void verifyOwnership(BookingResponse booking, UserDetails userDetails) {  
    User patient = userService.findByEmail(userDetails.getUsername());  
    if (!booking.getPatientUserId().equals(patient.getId())) {  
        throw new ResourceNotFoundException("Booking not found");  
    }  
}
```

Why 404 Not 403?

Resource hiding security pattern

Scenario: Patient A tries to read Patient B's booking 123

ALTERNATIVE 1: Return 403 Forbidden

Message: "You don't have access to this booking"

Problem: Confirms booking 123 EXISTS - just not yours

Attacker can enumerate: try 1, 2, 3, ..., 1000

"found one" (403) vs "doesn't exist" (404)

Now attacker knows which booking IDs are valid!

ALTERNATIVE 2: Return 404 Not Found (OUR CHOICE)

Message: "Booking not found"

Doesn't reveal whether it exists or belongs to someone else

Enumeration attacks become useless

This is called RESOURCE HIDING

Always prefer 404 over 403 when the resource exists but isn't accessible.

24. SecurityConfig Strategy

Secure by Default

Layered security

LAYER 1 (SecurityConfig):

- Explicitly permit-all the public endpoints
- `.anyRequest().authenticated()` catches everything else

LAYER 2 (@PreAuthorize on Controllers):

- Enforces specific roles
- Class-level annotation covers all methods

WHEN ADDING NEW ENDPOINTS:

Phase 4 added 18 endpoints, ZERO changes to SecurityConfig needed
Default = authenticated; @PreAuthorize handles roles
Cannot accidentally expose a new endpoint

INTERVIEW POINT » Interview Q: How do you design security to be additive when new features are added?

A: Two-layer approach with secure defaults. First, Spring Security configuration explicitly permits public endpoints and ends with `.anyRequest().authenticated()` - new endpoints are authenticated by default; you must opt-out for public access. Second, role-based authorization happens via `@PreAuthorize` annotations on controllers, typically at class level. When adding new features like Phase 4's booking endpoints, I don't need to touch SecurityConfig - the catch-all rule already protects them, and `@PreAuthorize` enforces role-specific access. This 'secure by default' design prevents the classic mistake of forgetting to secure a new endpoint.

PART VII · TESTING

25. End-to-End Setup Journey

Testing Phase 4 requires complete setup spanning Phase 1, 2, 3, and 4. The full journey:

#	Step	Endpoint	Phase
1	Register Admin user	POST /auth/register	1
2	Register Patient user	POST /auth/register	1
3	Register Doctor user	POST /auth/register	1
4	Login all 3, save tokens	POST /auth/login	1
5	Doctor creates profile	POST /doctors/me	2
6	Admin approves doctor	PUT /admin/doctors/{id}/approve	2
7	Admin creates slot config	POST /admin/doctors/{id}/slot-config	3
8	Admin generates slots	POST /admin/doctors/{id}/slots/generate	3
9	Patient books a slot	POST /me/bookings	4
10	Verify reference format	GET /me/bookings/{id}	4
11	Patient cancels	POST /me/bookings/{id}/cancel	4
12	Doctor completes booking	POST /doctors/me/bookings/{id}/complete	4
13	View history timeline	GET /me/bookings/{id}/history	4

TIP » Save credentials and tokens in Postman environment variables. Never paste tokens in shared messages - they're temporary passwords.

26. Smoke Test 1: Create Booking

Request

Request

POST http://localhost:8081/api/v1/me/bookings
Authorization: Bearer {{patient_token}}
Content-Type: application/json

```
{
  "slotId": 1,
  "notes": "Routine checkup."
}
```

Expected: 201 Created

Expected response

```
{
  "success": true,
  "message": "Booking created",
  "data": {
    "id": 1,
    "bookingReference": "DR2-20260615-001",    // YOUR generator!
    "status": "BOOKED",
    "patientUserId": 6,
    "patientName": "Patient Two",
    "slotId": 1,
    "slotDate": "2026-06-15",
    "slotStartTime": "10:00:00",
    "doctorName": "Dr. Two",
    "notes": "Routine checkup.",
    "createdAt": "..."
  }
}
```

What Just Happened

- Patient authenticated via JWT
- Pessimistic write lock acquired on slot
- Slot status validated (AVAILABLE, not expired)
- Capacity validated ($1 < 2$)
- Duplicate booking check passed
- Booking reference generated
- Booking entity saved
- Slot.current_bookings incremented (0 -> 1)
- BookingHistory entry created (action=CREATED)
- Transaction committed atomically, lock released

27. Smoke Test 2: Reference Format

Verify the booking reference matches the DR{doctorId}-{yyyyMMdd}-{sequence} format. From Test 1's response, the reference should be 'DR2-20260615-001':

Format verification

DR2-20260615-001

```
  ^      ^      ^
  |      |      +-- 001 = first booking for this (doctor, date)
  |      +----- 20260615 = June 15, 2026
  +----- DR2 = Doctor ID 2
```

Verify in MySQL:

```
SELECT id, booking_reference, status FROM bookings WHERE id = 1;
Result: 1, DR2-20260615-001, BOOKED
```

28. Smoke Test 3: Cancellation Flow

Variant A: Normal Cancellation (Far Future Slot)

Cancel request

POST http://localhost:8081/api/v1/me/bookings/1/cancel
Authorization: Bearer {{patient_token}}

```
{
  "reason": "Schedule conflict, will rebook later"
}
```

Expected: 200 OK

Cancel response

```
{
  "success": true,
  "message": "Booking cancelled",
  "data": {
    "id": 1,
    "status": "CANCELLED",
    "cancellationReason": "Schedule conflict, will rebook later",
    "cancelledByUserId": 6,
    "cancelledAt": "..."
  }
}
```

Variant B: Test 2-Hour Rule (Slot in 30 minutes)

2-hour rule verification

SCENARIO: Try to cancel a booking for a slot starting in 30 minutes.

Request: same POST /me/bookings/{id}/cancel
Expected: 400 Bad Request

```
{
  "success": false,
  "status": 400,
  "error": "Bad Request",
  "message": "Cannot cancel within 2 hours of the appointment.
              Please contact the clinic directly."
}
```

To set up this test:

1. Generate slots starting in <2 hours from now
2. Book one
3. Try to cancel -> should fail with the message above

This proves the 2-hour rule is enforced for patients.

Side Effects of Cancellation

- Booking.status = CANCELLED, audit fields populated
- Slot.current_bookings decremented
- Slot.status = AVAILABLE (if was FULLY_BOOKED)
- BookingHistory CANCELLED row inserted
- Slot is BOOKABLE AGAIN by another patient!

29. Smoke Test 4: Doctor Completes

Setup

This test requires a booking whose slot's end time has PASSED. The service throws `BadRequestException` if you try to complete a future booking. Either wait for a slot to pass naturally, or create a slot in the past via direct DB insert for testing.

Request

Complete request

POST `http://localhost:8081/api/v1/doctors/me/bookings/2/complete`

Authorization: `Bearer {{doctor_token}}`

Content-Type: `application/json`

```
{
  "notes": "Patient examined. Prescribed paracetamol 500mg twice daily."
}
```

Expected: 200 OK

Complete response

```
{
  "success": true,
  "message": "Booking completed",
  "data": {
    "id": 2,
    "status": "COMPLETED",
    "completedAt": "...",
    "notes": "Original patient notes\n[Doctor notes]: Patient examined..."
  }
}
```

Common Error: Slot Not Yet Passed

Expected error and fix

```
{
  "success": false,
  "status": 400,
  "message": "Cannot complete booking before its slot time"
}
```

FIX: Wait for slot time to pass, OR temporarily update slot date in DB:
`UPDATE doctor_slots SET slot_date = '2026-06-01' WHERE id = X;`

30. Smoke Test 5: View History

History request

GET <http://localhost:8081/api/v1/me/bookings/1/history>
Authorization: Bearer {{patient_token}}

Expected: 200 OK

Full history response

```
{
  "success": true,
  "data": {
    "booking": {
      "id": 1,
      "bookingReference": "DR2-20260615-001",
      "status": "CANCELLED",
      ...
    },
    "history": [
      {
        "id": 1,
        "bookingId": 1,
        "oldStatus": null,
        "newStatus": "BOOKED",
        "action": "CREATED",
        "performedByUserId": 6,
        "performedByUserEmail": "patient2@medibook.com",
        "performedAt": "2026-06-13T12:29:46"
      },
      {
        "id": 2,
        "bookingId": 1,
        "oldStatus": "BOOKED",
        "newStatus": "CANCELLED",
        "action": "CANCELLED",
        "performedByUserId": 6,
        "notes": "Schedule conflict, will rebook later",
        "performedAt": "..."
      }
    ]
  }
}
```

What This Proves

- BookingHistory entries created correctly
- old_status and new_status captured per row
- performed_by user info enriched (batch query!)
- Cancellation reason captured in notes
- Composite Detail+History response works
- Timeline is chronologically ordered

31. Common Errors & Solutions

Error	Likely Cause	Solution
403 Forbidden	Wrong token type	Use the role-matching token
401 Unauthorized	Token expired	Re-login to get fresh token
500 Internal Server	Check IntelliJ console	Console has the real stack trace
405 Method Not Allowed	Wrong HTTP method	Approve is PUT not POST
400 Validation Failed	Field name mismatch	Read error messages for exact field names
404 Booking not found	Patient accessing another's booking	Resource hiding pattern - intended!
Empty slot list	Phase 3 slots expired	Generate fresh slots for future dates
BCrypt hash failed	Password reset attempt	Register a fresh user instead
500 on approve	Maybe service bug	Paste console stack trace to diagnose
Cannot cancel within 2h	Slot too close	Expected behavior - the rule works!
Cannot complete before slot time	Future booking	Wait or backdate slot for testing

TIP » The IntelliJ console is your best friend for debugging. The Postman response is intentionally vague (security). Real errors are in the application logs.

PART VIII · REFERENCE

32. All 18 Endpoints Cheat Sheet

Patient (7 endpoints)

Method	Endpoint	Purpose
POST	/me/bookings	Create new booking
GET	/me/bookings	List own bookings (paginated)
GET	/me/bookings/{id}	View own booking detail
GET	/me/bookings/{id}/history	View timeline
POST	/me/bookings/{id}/cancel	Cancel (2-hour rule)
GET	/me/bookings/reference/{ref}	Lookup by reference

Doctor (6 endpoints)

Method	Endpoint	Purpose
GET	/doctors/me/bookings	List bookings on own slots
GET	/doctors/me/bookings/{id}	View detail
GET	/doctors/me/bookings/{id}/history	View timeline
POST	/doctors/me/bookings/{id}/cancel	Cancel (no time limit)
POST	/doctors/me/bookings/{id}/complete	Mark COMPLETED
POST	/doctors/me/bookings/{id}/no-show	Mark NO_SHOW

Admin (5 endpoints)

Method	Endpoint	Purpose
GET	/admin/bookings/{id}	View any booking
GET	/admin/bookings/{id}/history	View any timeline
GET	/admin/bookings/reference/{ref}	Lookup by reference
GET	/admin/patients/{id}/bookings	Patient's bookings

Method	Endpoint	Purpose
GET	/admin/doctors/{id}/bookings	Doctor's bookings
POST	/admin/bookings/{id}/cancel	Admin override cancel

33. Interview Q&A; for Phase 4

O1: How do you prevent double-booking?

INTERVIEW POINT » Three-layer defense. Layer 1: pessimistic write lock on the slot row during booking critical section - other transactions wait at the database level. Layer 2: @Version on entities for optimistic locking on general updates. Layer 3: UNIQUE constraint on (patient_user_id, slot_id) at the database catches duplicates even if application logic has bugs. The pessimistic lock prevents the storm, optimistic catches edge cases, and the DB constraint is the final safety net.

O2: Why have both @Version and @Lock?

INTERVIEW POINT » Different operations have different conflict profiles. The booking creation critical section has high contention (many patients hitting the same slot) - pessimistic locking is best there. General entity updates have low contention - optimistic locking is faster. Using both gives the best of each: serialized hot paths plus efficient general updates.

O3: Explain your audit trail design.

INTERVIEW POINT » Append-only BookingHistory table. Each row captures one state transition: old_status, new_status, action (CREATED/CANCELLED/etc.), performer, timestamp, notes. Never updated, never deleted - immutable historical record. Storing both old and new statuses (slight denormalization) makes queries trivial: 'all CANCELLED transitions' is a simple WHERE clause instead of LAG window functions. Append-only design eliminates write-write conflicts so no locking needed on the history table.

O4: How do you handle the N+1 query problem?

INTERVIEW POINT » Batch loading. Collect foreign keys into a Set (deduplicating), fetch all related entities in one query with findAllById, build a lookup map, then map entities using O(1) lookups. For our history responses, this turns 101 queries (1 + N) into 2 queries - 50x faster. Alternatives are JOIN FETCH in JPQL or @EntityGraph annotations; the batch approach gives the most control when you need just specific fields.

O5: Why role-based cancellation rules?

INTERVIEW POINT » Different roles have different responsibilities. Patients shouldn't cancel last-minute (no-show penalty for doctors), so 2-hour rule. But doctors MUST be able to cancel anytime for medical emergencies. Admins need overrides for system issues. Single service method handles all three with explicit role parameter from the controller - keeps business logic centralized while letting each controller signal its caller's role authoritatively.

O6: Why design a custom reference format?

INTERVIEW POINT » Self-descriptive identifiers. DR5-20261209-001 tells you doctor, date, and sequence at a glance. UUIDs are opaque - support staff can't read them. The format is sortable, debuggable, compact (20 chars vs 36 for UUID), and naturally encodes context. The trade-off is concurrency handling for the sequence, solved with retry logic and UNIQUE constraint catching collisions. For 50 bookings per doctor per day, count-based generation is trivially fast.

O7: Resource hiding vs explicit forbidden errors?

INTERVIEW POINT » Always 404 'Not Found' instead of 403 'Forbidden' for unauthorized resource access. Returning 403 confirms the resource exists - attackers can enumerate IDs (1, 2, 3, ...) to discover valid resource IDs. 404 doesn't reveal whether the resource exists or just isn't accessible. Combined with rate limiting and audit logging, this hardens against enumeration attacks. Same pattern used by GitHub for private repos.

34. Phase 5 Preview: Async Notifications

What's Coming Next

Phase 5 adds asynchronous communication. When a booking is created, the patient should get email/SMS confirmation. When cancelled, the doctor should be notified. These actions should NOT slow down the booking transaction - notifications are asynchronous.

New Concepts You'll Learn

- **@Async methods:** Run notifications in background threads
- **Event-driven architecture:** Publish BookingCreated events
- **Spring Application Events:** Decouple sender from receivers
- **Email integration:** SendGrid or Mailgun
- **SMS integration:** Twilio
- **Scheduled jobs:** @Scheduled annotations for reminders
- **Retry & circuit breakers:** Resilient external integrations

Architecture Sneak Peek

Event-driven Phase 5 architecture

BookingService creates booking

```
|  
+--> Publish BookingCreatedEvent (sync, just adds to queue)  
|  
+--> Returns response (FAST - no waiting!)
```

Meanwhile, async:

```
+--> EmailListener handles BookingCreatedEvent -> send email  
+--> SmsListener handles BookingCreatedEvent -> send SMS  
+--> AnalyticsListener handles BookingCreatedEvent -> log metrics
```

Each listener runs in its own thread, fail-independently.

PHASE 4 COMPLETE!

Production-grade booking system with:
Pessimistic + Optimistic locking · State machines
Rich audit trails · Role-based rules
Smart reference generation · Defense in depth

Repository: github.com/VIJAYKUMAR-G-CHATTARAKI/MediBook

4 of 11 phases complete - ready for Phase 5!